Projektszenario

Sie arbeiten als Junior-Entwickler in einem Softwareunternehmen. Ein Kunde benötigt ein kleines Konsolenprogramm zur Verwaltung von Bestellungen. Ihr Teamleiter erlaubt Ihnen, eine KI als Unterstützung zu nutzen – jedoch nur mit fachlicher Prüfung.

Phase 1 – Code-Generierung (mit KI)

Aufgabe

Lassen Sie sich von einer KI Ihrer Wahl ein Konsolenprogramm in **Python oder Java** erstellen, das:

- Bestellungen speichert
- Produktname
- Einzelpreis
- Menge
- Gesamtpreis berechnet
- Mehrere Bestellungen verwalten kann
- Gesamtsumme aller Bestellungen berechnet

Vorgaben:

- Nutzen Sie einen strukturierten Prompt.
- Speichern Sie den Prompt ab.
- Übernehmen Sie den Code nicht ungeprüft.
- Verbessern Sie ggf. den Code.
- Markieren Sie eigene Veränderungen.

Erstell mir als Junior-Entwickler, ein in Python geschriebenes, kleines und einfaches Konsolenprogramm zur Verwaltung von Bestellungen. In dem Programm sollen folgende Funktionen und Informationen mit enthalten sein:

1. Die Bestellungen speichern können
2. Der Produktname
3. Der Einzelpreis der Produkte
4. Die Menge der bestellten Produkte
5. Den Gesamtpreis aus allen bestellten Produkten berechnen
6. Mehrere und unterschiedliche Bestellungen von Personen verwalten können
7. Gesamtsumme aller Bestellungen berechnen

Erklär mir dabei bei jedem Codeblock, was dieser bewirken soll.

Fach	ZQ	BBS I Berufsbildende Schulen I des Landkreises Lüneburg	Klasse:
Thema	Prompting - KI als Entwicklungsassistent im Berufsalltag		
Lehrkraft	wd	Datum:	

Phase 2 – Debugging

Zusätzlich erhalten Sie folgenden fehlerhaften Code:

```python
def berechne_gesamtpreis(preis, menge):
    return preis * menge
```

```
    return preis * menge

preis = input("Preis eingeben: ")
menge = input("Menge eingeben: ")

gesamt = berechne_gesamtpreis(preis, menge)

print("Gesamtpreis:", gesamt)
```

Aufgabe

1. Analysieren Sie das Problem.
2. Lassen Sie sich von der KI beim Debugging helfen.
3. Erklären Sie dabei:
   - Warum tritt der Fehler auf?
   - Was ist der Unterschied zwischen String und Integer?
4. Verbessern Sie den Code.

1. Problem liegt in Zeile (2 da hier die Strings multipliziert werden miteinander) 4 und 5. Hier werden Strings erstellt keine Integer. Lösungsweg= Die Eingabe als Float definieren, da ja auch Nachkommazahlen angegeben werden können.
2. Keine Hilfe notwendig gewesen
3. Strings werden miteinander multipliziert -> Führt zu einem Fehler. Die Eingaben in Zeile 4 und 5 müssen als Float definiert werden.

   Strings sind Texte und Integer sind Ganzzahlen

```
def berechne_gesamtpreis(preis, menge):
    return preis * menge
while True:
    try:
        preis = float(input("Preis eingeben: "))
        menge = int(input("Menge eingeben: "))

    except ValueError:
        print("Du musst eine Zahl eingeben!")
        continue

    if preis < 0 or menge < 0:
        print("Die Zahlen müssen positiv sein!")
        continue

    gesamt = berechne_gesamtpreis(preis, menge)
    break

print(f"Gesamtpreis: {gesamt:.2f}€")
```

Phase 3 – Dokumentation

Erstellen Sie eine kurze technische Dokumentation (1–2 Seiten):

Inhalte:

- Programmbeschreibung
- Funktionsweise
- Verwendete Variablen
- Programmablauf
- Hinweise zur Erweiterung
- Screenshot des Programms

Zusatz:
Lassen Sie sich von der KI eine Dokumentation erzeugen und verbessern diese fachlich.

Reflexion:

- Was war ungenau?
- Wo mussten Sie nachbessern?

Phase 4 – Testfall-Erstellung

Aufgabe

Erstellen Sie mindestens 5 Testfälle:

Testfall Eingabe Erwartetes Ergebnis Tatsächliches Ergebnis

Beispiele:

- Normale Werte
- 0 als Menge
- Negative Werte
- Sehr große Zahlen
- Falsche Eingaben

Meine technische Dokumentation
Dieses Programm berechnet den **Gesamtpreis** eines Produkts. Dafür gibt der Benutzer einen **Preis** und eine **Menge** ein.
Anschließend wird beides miteinander multipliziert und das Ergebnis ausgegeben.
Am Anfang gibt es die Funktion berechne_gesamtpreis(preis, menge).
Diese Funktion hat die Aufgabe, den Preis mit der Menge zu multiplizieren. Das Ergebnis wird dann an das Hauptprogramm zurückgegeben.
Danach startet eine while True-Schleife.
Die Schleife sorgt dafür, dass der Benutzer so lange Eingaben machen kann, bis diese korrekt sind.
Innerhalb der Schleife werden die Werte mit input() eingegeben:

- preis wird als float gespeichert, also als Kommazahl
- menge wird als int gespeichert, also als ganze Zahl

Mit try und except ValueError wird geprüft, ob der Benutzer wirklich Zahlen eingibt.
Falls zum Beispiel ein Wort statt einer Zahl eingegeben wird, erscheint die Meldung:
Du musst eine Zahl eingeben!
Danach beginnt die Schleife wieder von vorne.
Anschließend wird mit einer if-Bedingung geprüft, ob preis oder menge kleiner als 0 sind.
Negative Zahlen sind hier nicht erlaubt, deshalb wird in diesem Fall die Meldung
Die Zahlen müssen positiv sein!
ausgegeben und die Schleife wiederholt.
Sind beide Eingaben gültig, wird die Funktion berechne_gesamtpreis(preis, menge) aufgerufen.
Das Ergebnis wird in der Variable gesamt gespeichert. Danach wird die Schleife mit break beendet.
Zum Schluss wird der berechnete Gesamtpreis mit print() ausgegeben.

Durch :.2f wird sichergestellt, dass das Ergebnis immer mit zwei Nachkommastellen angezeigt wird, zum Beispiel 12.50€.

```python
def berechne_gesamtpreis(preis, menge):  1 usage  new *
    return preis * menge
while True:
    try:
        preis = float(input("Preis eingeben: "))
        menge = int(input("Menge eingeben: "))

    except ValueError:
        print("Du musst eine Zahl eingeben!")
        continue

    if preis < 0 or menge < 0:
        print("Die Zahlen müssen positiv sein!")
        continue

    gesamt = berechne_gesamtpreis(preis, menge)
    break

print(f"Gesamtpreis: {gesamt:.2f}€")
```

KI Dokumentation:

Technische Dokumentation

Python-Programm zur Berechnung eines Gesamtpreises
mit integrierter Eingabeprüfung als Erweiterung

Ziel des Programms: Der Benutzer gibt Preis und Menge ein. Anschließend berechnet das Programm den Gesamtpreis und gibt ihn formatiert in Euro aus.

1. Programmbeschreibung

Das Programm dient zur Berechnung eines Gesamtpreises aus zwei Eingabewerten: **Preis** und **Menge**. Die eigentliche Berechnung wird in der Funktion *berechne_gesamtpreis(preis, menge)* durchgeführt. **Im erweiterten Stand** enthält das Programm zusätzlich eine Eingabeprüfung, damit nur gültige und positive Zahlen verarbeitet werden.

2. Funktionsweise

- Die Funktion multipliziert den Einzelpreis mit der eingegebenen Menge.
- Über eine while-Schleife wird die Eingabe so lange wiederholt, bis beide Werte korrekt sind.
- Mit try/except wird ein ValueError abgefangen, falls keine Zahl eingegeben wird.
- Zusätzlich prüft das Programm, ob Preis oder Menge negativ sind.
- Erst nach erfolgreicher Prüfung wird der Gesamtpreis berechnet und ausgegeben.

3. Verwendete Variablen

Die wichtigsten Variablen und ihre Aufgaben sind in der folgenden Tabelle zusammengefasst:

Variable	Datentyp	Bedeutung
preis	float	Speichert den eingegebenen Einzelpreis.
menge	int	Speichert die eingegebene Stückzahl.
gesamt	float	Enthält das Ergebnis aus Preis × Menge.

4. Programmablauf

1. Die Funktion zur Berechnung des Gesamtpreises wird definiert.
2. Das Programm fordert den Benutzer zur Eingabe von Preis und Menge auf.
3. Bei falscher Eingabe wird eine Fehlermeldung ausgegeben und die Eingabe wiederholt.
4. Bei negativen Werten erscheint ebenfalls eine Meldung und die Schleife startet erneut.
5. Sind beide Werte gültig, wird der Gesamtpreis berechnet.
6. Am Ende wird das Ergebnis formatiert mit zwei Nachkommastellen ausgegeben.

5. Hinweise zur Erweiterung

Mit diesem Punkt ist die bereits umgesetzte Erweiterung gemeint. Im Vergleich zum Ursprungscode wurde das Programm robuster gemacht:

Aspekt	Ursprungscode	Erweiterter Code
Falsche Eingaben	Führt bei Text statt Zahl zu einem Fehler.	try/except fängt den Fehler ab und fordert eine neue Eingabe an.
Negative Werte	Werden nicht geprüft.	Negative Zahlen werden erkannt und mit Hinweis abgelehnt.
Benutzerführung	Nur eine einmalige Eingabe.	while True wiederholt die Eingabe, bis gültige Werte vorliegen.

6. Screenshot des Ursprungsprogramms

Die folgende Abbildung zeigt den ursprünglichen Programmcode ohne die spätere Eingabeprüfung.

```python
def berechne_gesamtpreis(preis, menge):
    return preis * menge

preis = input("Preis eingeben: ")
menge = input("Menge eingeben: ")
```

```
gesamt = berechne_gesamtpreis(preis, menge)

print("Gesamtpreis:", gesamt)
```

Abbildung 1: Ursprungscode zur Berechnung des Gesamtpreises

Reflexion:

- Was war ungenau?
- Wo mussten Sie nachbessern?

Es wurde alles genau beschrieben und verständlich formuliert. Allerdings ist der Inhalt „Hinweise zur Erweiterung" nicht optimal formuliert wurden. Die KI hat es tabellarisch dargestellt, ich hätte hier einen Fließtext bevorzugt - Aber ist Grundsätzlich nicht falsch.

Abschlussreflexion (schriftlich)

Beantworten Sie bitte folgende Fragen:

1. Wo hat die KI euch gut unterstützt?
2. Wo war sie fehlerhaft oder ungenau?
3. Würdet ihr KI-Code direkt in ein Kundenprojekt übernehmen?
4. Wer trägt die Verantwortung für Fehler?
5. Was habt ihr über KI im Entwicklungsprozess gelernt?

1. Beim Verfassen der Dokumentation hat die KI mir immens geholfen. (Für den Code eher weniger)
2. In der Dokumentation waren zwar grundsätzlich keine Fehler vorhanden, aber einige Inhalte hätten anders bzw. besser formuliert werden können.
3. Natürlich würde ich KI generierten Code nicht direkt in Kundenprojekte übernehmen. Man kann und sollte KI für solche Fälle nutzen, aber dies sollte nur als Unterstützung dienen und auch nur, wenn man die Ausgaben/Antworten der KI kritisch hinterfragt und überprüft.
4. Die Verantwortung trägt hauptsächlich derjenige, der den KI-Code letzten Endes in das Kundenprojekt übernimmt.
5. KI kann in vielen Angelegenheiten eine große Unterstützung sein. Trotzdem sollte man ihre Antworten nicht einfach ungeprüft übernehmen, sondern sie immer kritisch hinterfragen und bei Bedarf kontrollieren, korrigieren oder anpassen.